

SISTEMA DE DIAGNÓSTICO DE SAÚDE DE TRANSFORMADORES ELÉTRICOS

Projeto Integrador — Microcontroladores

Tema 2: Manutenção Preditiva e Diagnóstico Operacional via IoT

- 
- P1** Hardware & Condicionamento de Sinal
 - P2** Firmware Base (Sensores & Tempo Real)
 - P3** DSP & Algoritmos de Análise
 - P4** Comunicação IoT (MQTT & WiFi)
 - P5** IHM Python (Dashboard & Alertas)
 - P6** Diagnóstico, Relatórios & Docs

Contents

1	Introdução e Motivação	3
1.1	Contextualização	3
1.2	Objetivos do Projeto	3
2	Fundamentação Teórica	4
2.1	Princípio de Funcionamento do Transformador	4
2.2	Conceitos e Fenômenos Físicos Fundamentais	5
2.2.1	Fluxo Magnético (Φ)	5
2.2.2	Saturação Magnética	5
2.2.3	Correntes de Foucault (Eddy Currents)	6
2.2.4	Histerese Magnética	6
2.2.5	Curto Parcial entre Espiras	7
2.3	Fenômenos Diagnósticos Monitorados	7
2.3.1	Corrente de Inrush	7
2.3.2	Gradiente Térmico e Eficiência	8
2.3.3	Assinatura Vibracional por Magnetostrição	8
3	Arquitetura do Sistema	9
3.1	Visão Geral por Camadas	9
3.2	Fluxo de Dados	10
3.3	Tópicos MQTT	10
4	Hardware	11
4.1	ESP32	11
4.2	SCT-013 — Transformador de Corrente Não Invasivo	12
4.2.1	Circuito de Condicionamento do SCT-013	12
4.3	DS18B20 — Sensor de Temperatura (Sonda Blindada)	13
4.4	MPU6050 — Acelerômetro + Giroscópio	13
5	Firmware ESP32	13
5.1	Princípio Fundamental: Código Não Bloqueante	13
5.2	Deteção do Inrush Current	14
5.3	FFT para Análise Vibracional (120 Hz)	15
5.4	Publicação MQTT com JSON	16

6	Software de Supervisão (IHM Python)	17
6.1	Arquitetura do Software	17
6.2	Dashboard e Visualização em Tempo Real	17
6.3	Módulo de Diagnóstico com Limites de Controle	18
6.4	Datalogger e Geração de Relatório PDF	19
7	Divisão de Tarefas (6 Pessoas)	20
8	Cronograma de Entregas	22
	Referências	22

1 Introdução e Motivação

1.1 Contextualização

Transformadores são componentes críticos em qualquer instalação elétrica industrial ou comercial. Uma falha não detectada num transformador pode resultar em interrupção de produção, danos irreversíveis a equipamentos conectados e riscos de incêndio por arco elétrico. No Brasil, a manutenção corretiva (consertar após a falha) ainda é a prática dominante em pequenas e médias instalações, em grande parte por falta de instrumentação acessível.

Sistemas comerciais de monitoramento contínuo de transformadores — como os oferecidos por empresas como ABB, Siemens e Schneider Electric — custam entre R\$ 15.000 e R\$ 80.000 por ponto de medição, tornando-os inviáveis para a maioria das aplicações. O presente projeto propõe uma alternativa de baixo custo, construída sobre hardware acessível (ESP32, sensores de menos de R\$ 50,00 cada) com capacidade de comunicação IoT.

1.2 Objetivos do Projeto

O sistema a ser desenvolvido tem como objetivo central criar uma plataforma de **manutenção preditiva** para transformadores, capaz de:

- Monitorar continuamente as grandezas elétricas, térmicas e mecânicas do transformador;
- Detectar anomalias características de falhas em desenvolvimento (degradação de isolamento, afrouxamento de núcleo, sobrecarga, curto entre espiras);
- Transmitir os dados em tempo real via protocolo MQTT para um painel de controle supervisão;
- Gerar alertas automáticos com sugestões de intervenção técnica;
- Registrar histórico com rastreabilidade por *timestamp* para laudos técnicos.

A norma ABNT NBR 5356 e as diretrizes do IEEE C57 recomendam monitoramento contínuo de temperatura e corrente em transformadores de potência. Vibração anormal em 120 Hz e seus múltiplos indica diretamente afrouxamento das chapas do núcleo magnético (*magnetostrição*), um fenômeno que aumenta progressivamente até causar falha catastrófica se não tratado.

2 Fundamentação Teórica

2.1 Princípio de Funcionamento do Transformador

Um transformador é um dispositivo eletromagnético que transfere energia elétrica entre dois circuitos isolados por meio de indução magnética mútua. Seus componentes essenciais são:

- **Bobina primária (N_1 espiras):** Conectada à rede de alimentação (220 V). A corrente alternada cria um campo magnético variável.
- **Núcleo de ferro laminado:** Conduz o fluxo magnético Φ gerado pelo primário até a bobina secundária com perdas mínimas. As chapas de aço-silício são laminadas justamente para reduzir correntes de Foucault.
- **Bobina secundária (N_2 espiras):** O fluxo magnético induz uma tensão proporcional à relação de espiras.

A relação fundamental que governa o transformador é:

$$\frac{V_1}{V_2} = \frac{N_1}{N_2} = \frac{I_2}{I_1} \quad (1)$$

Variáveis da Equação 1:

V_1 (V) Tensão eficaz aplicada ao enrolamento primário — a tensão da rede elétrica (220 V);

V_2 (V) Tensão eficaz induzida no enrolamento secundário — a tensão de saída (12 V);

N_1 Número de espiras do enrolamento primário (lado da rede);

N_2 Número de espiras do enrolamento secundário (lado da carga);

I_1 (A) Corrente eficaz que circula no enrolamento primário;

I_2 (A) Corrente eficaz que circula no enrolamento secundário e é fornecida à carga.

A relação entre correntes aparece *invertida* em relação à tensão: se o secundário tem menos espiras e portanto menor tensão, ele fornece uma corrente maior na mesma proporção — a potência é conservada (descontadas as perdas). Matematicamente: se $V_2 = V_1/18,3$, então $I_2 = I_1 \times 18,3$. Para o transformador monitorado neste projeto (220 V / 12 V), a relação de espiras é $N_1/N_2 \approx 18,3$.

2.2 Conceitos e Fenômenos Físicos Fundamentais

Para compreender o que o sistema monitora e por que cada sensor está posicionado onde está, é necessário entender os fenômenos físicos que ocorrem dentro de um transformador em operação. Esta seção serve como referência rápida para todos os integrantes da equipe.

2.2.1 Fluxo Magnético (Φ)

Quando uma corrente elétrica alternada circula pelo enrolamento primário, ela cria um campo magnético variável ao redor do fio. O núcleo de ferro laminado tem a função de *concentrar e canalizar* esse campo até o enrolamento secundário, da mesma forma que um cano canaliza água. O fluxo magnético Φ (medido em Weber, Wb) é a grandeza que quantifica essa “quantidade de campo” que atravessa a seção transversal do núcleo.

A lei de Faraday estabelece que uma variação de fluxo no tempo induz uma tensão no secundário:

$$e = -N_2 \cdot \frac{d\Phi}{dt} \quad (2)$$

Variáveis da Equação 2:

e (V) Força eletromotriz (tensão) induzida no secundário;

N_2 Número de espiras do enrolamento secundário;

$d\Phi/dt$ (Wb/s) Taxa de variação do fluxo magnético no tempo;

“−” Sinal negativo: a tensão induzida se opõe à variação que a causou (Lei de Lenz).

Em outras palavras: sem variação de fluxo, não há tensão induzida. É por isso que transformadores só funcionam com corrente *alternada* — em corrente contínua o fluxo seria constante ($d\Phi/dt = 0$) e nada seria induzido no secundário.

2.2.2 Saturação Magnética

O núcleo de ferro não consegue conduzir fluxo magnético indefinidamente. Existe um limite físico chamado **saturação magnética**: acima de certo nível de campo aplicado (H), os domínios magnéticos do material já estão todos alinhados e o ferro simplesmente para de responder ao aumento de corrente.

Na prática, isso significa que:

- Abaixo da saturação: a relação entre corrente primária e fluxo é *linear*. O transformador funciona normalmente.

- Na saturação: o fluxo “trava” enquanto a corrente continua subindo sem limite. O resultado é um pico de corrente massivo no primário sem correspondência no secundário.

Um transformador saudável, ao ser energizado, entra brevemente em saturação e gera o pico de *inrush* (ver Seção 2.3.1) que decai suavemente em décimos de segundo. Se a saturação ocorre com *pico mais alto do que esperado* ou com *decaimento assimétrico*, isso indica degradação do núcleo ou falha na isolação primária. O SCT-013 no primário captura exatamente esse evento.

2.2.3 Correntes de Foucault (Eddy Currents)

O mesmo campo magnético variável que induz tensão no secundário também induz tensões no próprio núcleo de ferro. Como o ferro conduz eletricidade, essas tensões fazem circular correntes fechadas dentro do material — as chamadas **correntes de Foucault** (ou *eddy currents*). Essas correntes não fazem nenhum trabalho útil: elas apenas dissipam energia na forma de calor dentro do núcleo (**perdas no ferro**).

Para minimizá-las, o núcleo não é um bloco sólido de ferro, mas sim um empilhamento de **chapas finas isoladas entre si** (as lâminas de aço-silício). Ao dividir o núcleo em fatias, o caminho das correntes de Foucault é interrompido, reduzindo drasticamente as perdas.

Quando as chapas do núcleo se afrouxam por vibração ou degradação do verniz isolante, as correntes de Foucault aumentam e a amplitude de vibração em 120 Hz cresce. O MPU6050 detecta esse aumento de amplitude — é o sinal diagnóstico de que as chapas precisam de aperto mecânico.

2.2.4 Histerese Magnética

O núcleo de ferro não “esquece” instantaneamente o campo que recebeu. Quando a corrente primária inverte de sentido, o fluxo magnético não acompanha imediatamente — existe um atraso chamado **histerese**. A energia gasta para vencer esse atraso a cada ciclo também se dissipa como calor no núcleo (perdas por histerese).

Em condições normais, as perdas por histerese são constantes e previsíveis para uma dada temperatura de operação. Um aumento anormal de temperatura *sem* correspondente aumento de carga pode indicar que as propriedades magnéticas do núcleo degradaram (oxidação, temperatura excessiva prévia, envelhecimento), aumentando as perdas por histerese. O DS18B20 monitora exatamente essa variação térmica.

2.2.5 Curto Parcial entre Espiras

Uma das falhas mais perigosas e difíceis de detectar num transformador é o **curto parcial entre espiras adjacentes**: o verniz isolante que reveste o fio de cobre se degrada (por calor, vibração ou surtos de tensão), e algumas espiras entram em curto-circuito entre si, formando um “loop” de corrente fechado dentro da própria bobina.

Os efeitos são cumulativos e progressivos:

1. O número efetivo de espiras cai $\rightarrow$ a relação N_1/N_2 muda $\rightarrow$ a tensão de saída cai ligeiramente;
2. O “loop” em curto dissipa potência como calor localizado $\rightarrow \Delta T$ cresce;
3. O aquecimento acelera a degradação do verniz vizinho $\rightarrow$ mais espiras entram em curto $\rightarrow$ ciclo destrutivo.

O curto parcial entre espiras é detectado pela **combinação** de dois sinais: (1) gradiente térmico ΔT elevado para a carga atual, medido pelo DS18B20, e (2) queda na eficiência η , calculada pelos SCT-013. Nenhum dos dois sinais isolado é suficiente — a correlação entre eles é o diagnóstico.

2.3 Fenômenos Diagnósticos Monitorados

2.3.1 Corrente de Inrush

Quando um transformador é conectado à rede elétrica pela primeira vez (ou após uma desconexão), o núcleo de ferro não possui fluxo magnético residual uniforme. Dependendo do instante exato em que a chave é fechada (a fase da onda de tensão naquele momento), o núcleo pode ser forçado a atingir um nível de fluxo que *ultrapassa sua capacidade de saturação* nos primeiros semiciclos.

Como o núcleo saturado não oferece mais reatância (resistência ao fluxo), a corrente no primário sobe drasticamente limitada apenas pela resistência ôhmica do enrolamento — que é baixíssima. Isso gera o chamado **inrush current**: um pico de corrente que pode atingir **8 a 12 vezes** a corrente nominal, com duração típica de 0,1 a 0,3 segundos, decaindo exponencialmente conforme o núcleo “se magnetiza” de forma estável.

A análise do perfil de inrush revela o estado do núcleo:

- **Inrush normal**: Pico elevado com decaimento exponencial suave — o núcleo sai da saturação e o transformador entra em regime de corrente de magnetização estável (a corrente mínima que o primário consome mesmo sem carga, necessária para manter o fluxo no núcleo);

- **Saturação precoce do núcleo:** Inrush com pico desproporcionalmente alto para a carga — indica que a permeabilidade magnética do núcleo degradou (material oxidado, temperatura excessiva anterior, desmagnetização parcial);
- **Falha de isolamento inicial:** Assimetria no perfil de decaimento ou picos secundários — indica resistência parasita ou curto parcial na região da bobina.

2.3.2 Gradiente Térmico e Eficiência

Um transformador eficiente converte energia com perdas mínimas. As perdas existentes se dividem em dois grupos: **perdas no cobre** (aquecimento dos enrolamentos pela corrente, proporcional a I^2R) e **perdas no ferro** (histerese e correntes de Foucault no núcleo, aproximadamente constantes para uma dada tensão de alimentação). O rendimento é definido como:

$$\eta = \frac{P_{out}}{P_{in}} = \frac{V_2 \cdot I_2}{V_1 \cdot I_1} \quad (3)$$

Variáveis da Equação 3:

η Rendimento (eficiência) do transformador, adimensional (0 a 1, ou 0% a 100%);

P_{out} (W) Potência entregue à carga no secundário;

P_{in} (W) Potência consumida da rede pelo primário;

V_2 (V) Tensão eficaz no secundário (medida ou calculada);

I_2 (A) Corrente eficaz no secundário, lida pelo SCT-013 secundário;

V_1 (V) Tensão eficaz no primário (assumida como 220 V nominal);

I_1 (A) Corrente eficaz no primário, lida pelo SCT-013 primário.

Transformadores de boa qualidade operam com $\eta > 95\%$. Quando o gradiente térmico ΔT (diferença entre a temperatura do núcleo e a temperatura ambiente) cresce progressivamente sem que a corrente de carga I_2 tenha aumentado, isso significa que P_{in} está crescendo sem P_{out} acompanhar — ou seja, η está caindo. Este é o sinal diagnóstico de curto parcial entre espiras (ver Seção 2.3).

2.3.3 Assinatura Vibracional por Magnetostricção

Magnetostricção é a propriedade que materiais ferromagnéticos (como o ferro e o aço-silício do núcleo) têm de *mudar ligeiramente de dimensão* quando submetidos a um campo magnético. Quando o campo magnético aumenta, o material se expande; quando o campo diminui, o material se contrai. Isso ocorre em nível atômico: os domínios

magnéticos (regiões onde os átomos estão alinhados) se reorientam com o campo, e essa reorientação provoca uma deformação mecânica microscópica do cristal.

O campo magnético no núcleo varia na frequência da rede (60 Hz), mas a deformação mecânica ocorre tanto na subida quanto na descida do campo — ou seja, *duas deformações por ciclo*. Por isso a vibração mecânica resultante ocorre em $2 \times 60 = \mathbf{120\ Hz}$, independentemente do sentido do campo. Os harmônicos (240 Hz, 360 Hz) aparecem por conta das não-linearidades da curva de magnetização do ferro.

Em um transformador bem construído e com as chapas bem fixadas, essa vibração existe mas é pequena e estável. O problema ocorre quando:

- O verniz ou tinta que une as chapas do núcleo envelhece e perde aderência;
- Os parafusos de aperto do núcleo se afrouxam por vibração acumulada;
- O transformador opera em sobrecarga por longos períodos (maior campo magnético $\rightarrow$ maior deformação).

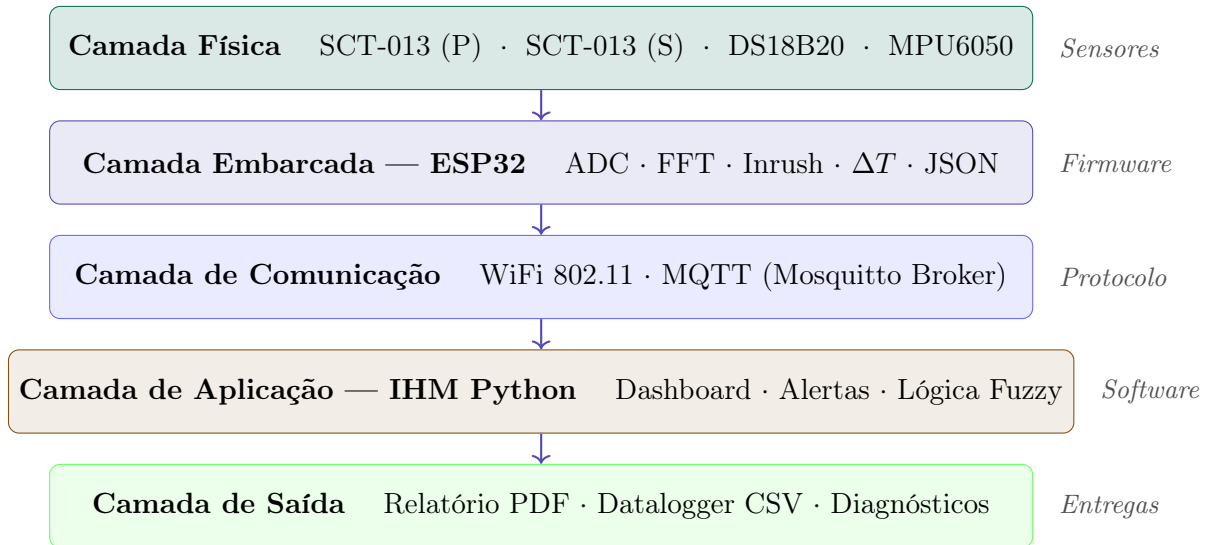
Com as chapas soltas, a deformação de magnetostricção se converte em vibração mecânica muito maior — as chapas literalmente “batem” entre si a 120 Hz. Esse fenômeno é o famoso **“zumbido” do transformador** que piora ao longo do tempo.

O MPU6050 captura a aceleração do chassi do transformador. A FFT do sinal de aceleração revela o espectro de frequências das vibrações. Um transformador saudável apresenta amplitude em 120 Hz dentro de limites esperados para sua potência nominal. Amplitude crescente indica afrouxamento das chapas — problema mecânico que agrava as perdas no ferro e, se não tratado, leva ao curto entre espiras por desgaste da isolamento.

3 Arquitetura do Sistema

3.1 Visão Geral por Camadas

O sistema é organizado em cinco camadas funcionais, do mundo físico ao relatório técnico:



3.2 Fluxo de Dados

O ciclo de operação completo do sistema pode ser descrito da seguinte forma:

1. Os sensores físicos capturam grandezas contínuas do transformador (corrente, temperatura, vibração) e as convertem em sinais elétricos condicionados para o ADC do ESP32.
2. O ESP32, operando em tempo real sem bloqueios (*non-blocking loop*), amostra os sinais periodicamente, aplica os algoritmos DSP e verifica limites de alarme.
3. Os resultados são serializados em JSON com *timestamp* e publicados nos respectivos tópicos MQTT via WiFi.
4. O broker Mosquitto distribui os pacotes ao software Python inscrito nos tópicos.
5. A IHM Python plota os dados em tempo real, aplica lógica de diagnóstico e aciona alertas visuais quando necessário.
6. O datalogger persiste todos os registros em CSV; ao final do período, um relatório PDF consolidado é gerado automaticamente.

3.3 Tópicos MQTT

A organização dos tópicos segue hierarquia semântica para facilitar inscrição seletiva e escalabilidade futura:

Tópico	Tipo de dado	Unidade
transformador/primario/corrente	Corrente eficaz (RMS)	A
transformador/primario/inrush	Flag + pico	A
transformador/secundario/corrente	Corrente eficaz	A
transformador/nucleo/temperatura	Temperatura absoluta	°C
transformador/nucleo/delta_t	Gradiente térmico	°C
transformador/vibracao/fft_120hz	Amplitude 120 Hz	g
transformador/vibracao/fft_240hz	Amplitude 240 Hz	g
transformador/status/alarme	JSON estruturado	—
transformador/status/heartbeat	<i>Timestamp</i> Unix	s

4 Hardware

4.1 ESP32

O ESP32 é o microcontrolador central do projeto. Suas especificações relevantes para esta aplicação são:

- **Processador:** Xtensa LX6 dual-core a 240 MHz;
- **ADC:** 12 bits, até 18 canais (atenção: GPIO34 e GPIO35 são *input-only* — ideais para ADC por não possuírem resistores de pull-up internos que distorcem leituras analógicas);
- **Comunicação:** WiFi 802.11 b/g/n integrado, I²C, SPI, UART;
- **Interrupções:** Todos os GPIOs suportam `attachInterrupt()`;
- **Timers de hardware:** 4 timers de 64 bits, essenciais para temporização precisa.

Atribuição de pinos:

GPIO	Periférico	Função
GPIO21	MPU6050 (SDA)	I ² C — dados
GPIO22	MPU6050 (SCL)	I ² C — clock
GPIO4	DS18B20 (DATA)	OneWire
GPIO34	SCT-013 Primário	ADC1 CH6 (leitura)
GPIO35	SCT-013 Secundário	ADC1 CH7 (leitura)
3V3	Todos os sensores	Alimentação
GND	Todos os sensores	Referência

4.2 SCT-013 — Transformador de Corrente Não Invasivo

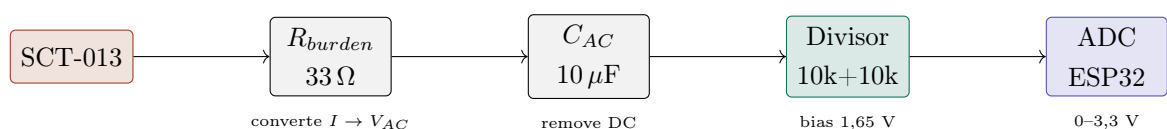
O SCT-013 é um **transformador de corrente não invasivo** (*split-core current transformer*). Seu funcionamento é baseado no mesmo princípio de indução eletromagnética do transformador principal que estamos monitorando:

- O fio do transformador monitorado passa pelo núcleo toroidal interno do SCT-013 — esse fio é o “enrolamento primário” do sensor (uma única espira, $N_1 = 1$);
- A corrente que circula nesse fio cria um campo magnético que induz uma corrente proporcional no enrolamento secundário interno do SCT-013, com muitas espiras ($N_2 \approx 2000$);
- Pela relação de transformadores: $I_{sensor} = I_{fio}/N_2$ — para 30 A no fio, a saída é $30/2000 = 15$ mA;
- A variante SCT-013-030 produz saída máxima de 1 A para 30 A no fio primário;
- A corrente de saída deve ser convertida em tensão por um resistor *burden* externo (ver circuito de condicionamento).

O sensor *abraça* o fio sem cortar ou interromper o circuito monitorado. O núcleo do SCT-013 é partido em duas metades que se encaixam ao redor do fio. Isso é essencial para o nosso projeto: podemos instalar e remover o sensor com o transformador em operação, sem desligar a instalação.

O ADC do ESP32 só aceita tensões entre 0 V e 3,3 V. O sinal do SCT-013 é AC centrado em 0 V — impossível de ler diretamente. O circuito de condicionamento abaixo é **obrigatório** para não danificar o ESP32.

4.2.1 Circuito de Condicionamento do SCT-013



O divisor resistivo (2×10 kΩ entre 3,3 V e GND) gera uma tensão de referência de 1,65 V, que serve como ponto central para o sinal AC. O capacitor de $10 \mu\text{F}$ faz o acoplamento AC, removendo qualquer offset DC. O resultado é um sinal AC centrado em 1,65 V, variando entre aproximadamente 0 e 3,3 V — dentro da faixa do ADC.

4.3 DS18B20 — Sensor de Temperatura (Sonda Blindada)

- **Interface:** OneWire (1 único fio de dados + GND + Vcc);
- **Resolução:** Até 12 bits (0,0625 °C por passo);
- **Faixa:** -55 °C a +125 °C;
- **Encapsulamento recomendado:** Sonda blindada em aço inox, adequada para contato direto com superfícies metálicas quentes;
- **Resistor pull-up:** Obrigatório, valor de 4,7 kΩ entre DATA e 3,3 V.

Posicionamento: A sonda deve ser fixada diretamente sobre o núcleo ou sobre a lateral da bobina com fita kapton, com uma camada fina de pasta térmica entre a sonda e a superfície para melhorar o contato térmico.

4.4 MPU6050 — Acelerômetro + Giroscópio

- **Interface:** I²C (endereço padrão: 0x68);
- **Acelerômetro:** 3 eixos, ±16 g de fundo de escala, 16 bits;
- **Giroscópio:** 3 eixos (não utilizado neste projeto);
- **Taxa de amostragem:** Configurável até 1 kHz — suficiente para capturar 120 Hz e harmônicos até a 4^a ordem (480 Hz) com folga;
- **Alimentação:** 3,3 V diretamente do ESP32.

Posicionamento: Parafusado ou preso com abraçadeira diretamente no chassi metálico do transformador, o mais rígido possível. Folgas mecânicas entre o sensor e o chassi atenuam as vibrações e comprometem a análise FFT.

5 Firmware ESP32

5.1 Princípio Fundamental: Código Não Bloqueante

O uso de `delay()` é **estritamente proibido** em qualquer parte do código. Durante um `delay()`, o ESP32 para completamente — deixa de amostrar sensores, perde eventos de interrupção e não mantém a conexão MQTT ativa. Todo controle de tempo deve usar `millis()` ou timers de hardware.

A estrutura correta do loop principal utiliza o padrão de *state machine* com `millis()`:

```
1 // Intervalos em milissegundos
2 const uint32_t INTERVALO_TEMPERATURA = 2000; // 2 s
3 const uint32_t INTERVALO_CORRENTE   = 50;   // 20 amostras/s
4 const uint32_t INTERVALO_MQTT       = 1000; // publicar a cada 1 s
5
6 uint32_t ultimaTemp      = 0;
7 uint32_t ultimaCorrente = 0;
8 uint32_t ultimaMQTT      = 0;
9
10 void loop() {
11     uint32_t agora = millis();
12
13     if (agora - ultimaCorrente >= INTERVALO_CORRENTE) {
14         ultimaCorrente = agora;
15         lerADC();        // SCT-013 primario e secundario
16         lerMPU6050();    // MPU6050 via I2C
17     }
18
19     if (agora - ultimaTemp >= INTERVALO_TEMPERATURA) {
20         ultimaTemp = agora;
21         lerDS18B20();    // temperatura OneWire
22         calcularDeltaT();
23     }
24
25     if (agora - ultimaMQTT >= INTERVALO_MQTT) {
26         ultimaMQTT = agora;
27         publicarMQTT();
28     }
29
30     mqtt.loop(); // manter conexao MQTT ativa (NON-BLOCKING)
31 }
```

Listing 1: Estrutura não bloqueante com `millis()`

5.2 Detecção do Inrush Current

O inrush ocorre no instante da energização. A detecção é feita monitorando a corrente do primário com alta frequência de amostragem e verificando se ultrapassa um limiar configurável:

```
1 #define LIMIAR_INRUSH_A  5.0 // ajustar para a corrente nominal do
   trafo
2 #define JANELA_INRUSH_MS 500 // janela de tempo para captura (ms)
```

```

3
4 bool   inrushDetectado = false;
5 float picoInrush      = 0.0;
6
7 void verificarInrush(float correnteAtual) {
8     static uint32_t tInicio = 0;
9     static bool     monitorando = false;
10
11     // Início do monitoramento ao detectar pico acima do limiar
12     if (!monitorando && correnteAtual > LIMAR_INRUSH_A) {
13         monitorando      = true;
14         tInicio          = millis();
15         inrushDetectado  = true;
16         picoInrush       = correnteAtual;
17     }
18
19     // Atualizar pico durante a janela
20     if (monitorando) {
21         if (correnteAtual > picoInrush) picoInrush = correnteAtual;
22         if (millis() - tInicio > JANELA_INRUSH_MS) monitorando = false;
23     }
24 }

```

Listing 2: Detecção de Inrush Current

5.3 FFT para Análise Vibracional (120 Hz)

A FFT é aplicada sobre um buffer de amostras do MPU6050. A FFT (*Fast Fourier Transform* — Transformada Rápida de Fourier) é um algoritmo que converte um sinal no domínio do tempo (sequência de leituras de aceleração ao longo do tempo) para o domínio da frequência (quais frequências estão presentes e com que amplitude). Para detectar 120 Hz com resolução adequada, o buffer deve ser dimensionado corretamente:

$$\Delta f = \frac{f_s}{N} \quad (4)$$

Variáveis da Equação 4:

Δf (Hz) Resolução espectral — a menor diferença de frequência que a FFT consegue distinguir. Quanto menor, mais preciso o espectro;

f_s (Hz) Frequência de amostragem — quantas leituras por segundo o MPU6050 fornece ao ESP32;

N Número de amostras no buffer — o tamanho da “janela” de tempo analisada. Deve ser potência de 2 para o algoritmo FFT funcionar eficiente-

mente (64, 128, 256, 512...).

Critério de Nyquist: Para capturar uma frequência f , a taxa de amostragem deve ser pelo menos $f_s \geq 2f$. Para capturar 240 Hz (segunda harmônica), precisamos de $f_s \geq 480$ Hz. Usamos $f_s = 500$ Hz com folga.

Para $f_s = 500$ Hz e $N = 256$ amostras: $\Delta f = 500/256 = 1,95$ Hz, resolução suficiente para isolar 120 Hz dos harmônicos adjacentes. A biblioteca `arduinoFFT` implementa o algoritmo eficientemente no ESP32:

```

1 #include <arduinoFFT.h>
2
3 const uint16_t AMOSTRAS = 256;
4 const float    FREQ_SAMP = 500.0; // Hz
5
6 double vReal[AMOSTRAS], vImag[AMOSTRAS];
7 arduinoFFT FFT = arduinoFFT(vReal, vImag, AMOSTRAS, FREQ_SAMP);
8
9 float calcFFT_120hz() {
10     // Preencher buffer com leituras do MPU6050 (eixo Z)
11     for (uint16_t i = 0; i < AMOSTRAS; i++) {
12         vReal[i] = lerAceleracaoZ(); // valor em g
13         vImag[i] = 0;
14         delayMicroseconds(1000000 / FREQ_SAMP); // pausa MICRO, nao
15             millis
16     }
17
18     FFT.Windowing(FFT_WIN_TYP_HAMMING, FFT_FORWARD);
19     FFT.Compute(FFT_FORWARD);
20     FFT.ComplexToMagnitude();
21
22     // Indice correspondente a 120 Hz: (120 / 500) * 256 = 61.44 ->
23     // indice 61
24     uint16_t idx120hz = (uint16_t)(120.0 / FREQ_SAMP * AMOSTRAS);
25     return vReal[idx120hz];
26 }

```

Listing 3: FFT com `arduinoFFT`

5.4 Publicação MQTT com JSON

```

1 #include <ArduinoJson.h>
2
3 void publicarLeitura(const char* topico, float valor, const char*
4     unidade) {
5     StaticJsonDocument<128> doc;
6     doc["ts"] = millis() / 1000;

```

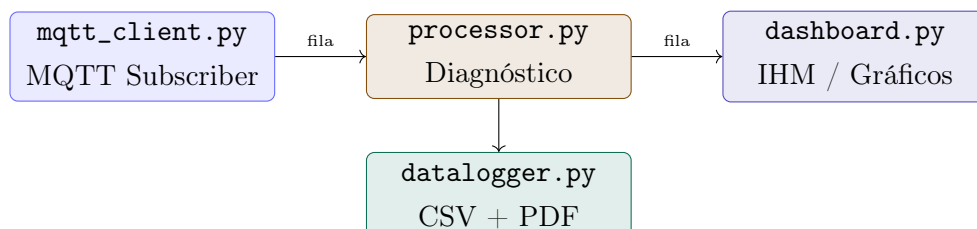
```
6   doc["valor"]    = valor;
7   doc["unidade"]  = unidade;
8
9   char payload[128];
10  serializeJson(doc, payload);
11  mqtt.publish(topico, payload, false); // QoS 0
12 }
13
14 void publicarAlarme(const char* tipo, const char* sev,
15                    float valor, float limite) {
16     StaticJsonDocument<256> doc;
17     doc["ts"]      = millis() / 1000;
18     doc["tipo"]    = tipo;
19     doc["sev"]     = sev;
20     doc["valor"]   = valor;
21     doc["limite"]  = limite;
22
23     char payload[256];
24     serializeJson(doc, payload);
25     mqtt.publish("transformador/status/alarme", payload, true); //
        retained
26 }
```

Listing 4: Serialização JSON e publicação MQTT

6 Software de Supervisão (IHM Python)

6.1 Arquitetura do Software

O software Python é organizado em módulos independentes que se comunicam via fila de dados (`queue.Queue`), permitindo que a thread de aquisição MQTT não bloqueie a thread de renderização do dashboard:



6.2 Dashboard e Visualização em Tempo Real

Recomenda-se o uso de **Streamlit** ou **NiceGUI** para o dashboard, com atualização por *websocket*. Cada variável monitorada deve ter:

- Gráfico de linha com janela deslizante dos últimos N segundos;
- Indicador LED virtual (verde/amarelo/vermelho) com base em limiares;
- Valor numérico atual com unidade.

```
1 import paho.mqtt.client as mqtt
2 import json, queue
3
4 fila_dados = queue.Queue()
5
6 TOPICOS = [
7     "transformador/primario/corrente",
8     "transformador/secundario/corrente",
9     "transformador/nucleo/temperatura",
10    "transformador/vibracao/fft_120hz",
11    "transformador/status/alarme",
12 ]
13
14 def ao_receber_mensagem(client, userdata, msg):
15     payload = json.loads(msg.payload.decode())
16     fila_dados.put({"topico": msg.topic, **payload})
17
18 client = mqtt.Client()
19 client.on_message = ao_receber_mensagem
20 client.connect("localhost", 1883)
21 for t in TOPICOS:
22     client.subscribe(t)
23 client.loop_start() # Thread separada, nao bloqueia
```

Listing 5: Subscriber MQTT com fila

6.3 Módulo de Diagnóstico com Limites de Controle

A lógica de diagnóstico compara os valores recebidos contra limiares configuráveis e gera sugestões de intervenção:

```
1 LIMITES = {
2     "temperatura": {"aviso": 70.0, "critico": 85.0},
3     "fft_120hz": {"aviso": 0.20, "critico": 0.45},
4     "delta_t_carga": {"aviso": 15.0, "critico": 25.0},
5 }
6
7 MENSAGENS_DIAGNOSTICO = {
8     "temperatura_critico":
9         "CRITICO: Temperatura acima de {}C. "
```

```

10     "Risco de curto parcial entre espiras. "
11     "Reduzir carga imediatamente.",
12     "fft_120hz_critico":
13         "MANUTENCAO: Vibração 120Hz fora do padrao ({:.3f} g). "
14         "Realizar aperto mecanico das chapas do nucleo.",
15     "delta_t_carga_aviso":
16         "AVISO EFICIENCIA: Gradiente termico elevado para a "
17         "carga atual. Monitorar degradacao da relacao Pin/Pout.",
18 }
19
20 def diagnosticar(dados: dict) -> list:
21     alertas = []
22     for grandeza, lims in LIMITES.items():
23         valor = dados.get(grandeza)
24         if valor is None:
25             continue
26         if valor >= lims["critico"]:
27             chave = f"{grandeza}_critico"
28         elif valor >= lims["aviso"]:
29             chave = f"{grandeza}_aviso"
30         else:
31             continue
32         msg = MENSAGENS_DIAGNOSTICO.get(chave, f"Alerta: {grandeza} = {
33             valor}")
34         alertas.append({"chave": chave, "valor": valor, "mensagem": msg
35 })
36     return alertas

```

Listing 6: Lógica de diagnóstico com limites de controle

6.4 Datalogger e Geração de Relatório PDF

O datalogger grava todos os registros em CSV com *timestamp* ISO 8601. A geração de PDF usa a biblioteca *reportlab*:

```

1 import csv, datetime, os
2
3 ARQUIVO_CSV = "historico_transformador.csv"
4 CABECALHO = ["timestamp", "topico", "valor", "unidade", "alarme"]
5
6 def gravar_registro(topico, valor, unidade, alarme=""):
7     agora = datetime.datetime.now().isoformat()
8     existe = os.path.exists(ARQUIVO_CSV)
9     with open(ARQUIVO_CSV, "a", newline="", encoding="utf-8") as f:
10         writer = csv.writer(f)
11         if not existe:

```

```

12     writer.writerow(CABECALHO)
13     writer.writerow([agora, topico, valor, unidade, alarme])

```

Listing 7: Datalogger CSV com timestamp

7 Divisão de Tarefas (6 Pessoas)

Cada integrante é dono de uma entrega clara e verificável, mas **P1 e P2 devem alinhar o pinout e a estrutura do loop() antes de começar**, e **P4 deve entregar o broker MQTT configurado na primeira semana** para que P5 e P6 possam testar com dados reais.

Pessoa	Responsabilidade	Entregas Verificáveis
P1 <i>Hardware</i>	Montagem do circuito físico, circuito de condicionamento dos dois SCT-013 (burden + divisor + capacitor), pinagem do ESP32, testes de bancada com multímetro.	Circuito montado na protoboard, sinal condicionado mensurável no osciloscópio ou plotter serial, sem risco ao ADC.
P2 <i>Firmware Base</i>	Código não-bloqueante com <code>millis()</code> , integração DS18B20 (OneWire) e SCT-013 (ADC), interrupções de hardware, leitura estável do MPU6050 via I ² C.	Leituras de temperatura, corrente e aceleração aparecendo no Serial Monitor com timestamp, sem uso de <code>delay()</code> .
P3 <i>DSP & Algoritmos</i>	Implementação da FFT no ESP32 (biblioteca <code>arduinoFFT</code>), extração da amplitude em 120 Hz e 240 Hz, detecção de Inrush Current, cálculo do gradiente térmico ΔT .	Flag de Inrush funcionando na energização, espectro vibracional com valor em 120 Hz extraído corretamente.

(continua na próxima página)

Pessoa	Responsabilidade	Entregas Verificáveis
P4 <i>IoT & MQTT</i>	Configuração do broker Mosquitto no computador da equipe, implementação do cliente MQTT no ESP32 (<code>PubSubClient</code>), serialização JSON com <i>timestamp</i> , teste de publicação e inscrição.	ESP32 publicando JSON nos tópicos definidos, testável com <code>mosquitto_sub</code> no terminal, sem perda de pacotes.
P5 <i>IHM Python</i>	Dashboard em Streamlit ou NiceGUI, subscriber MQTT em Python (<code>paho-mqtt</code>), gráficos em tempo real, indicadores LED virtuais (verde/amarelo/vermelho), pop-ups de alerta.	Painel funcionando com dados reais do broker, todas as variáveis plotadas, alertas visuais acionando corretamente.
P6 <i>Diagnóstico & Relatórios</i>	Lógica de diagnóstico (limites de controle + mensagens de intervenção), datalogger CSV com timestamp, geração de relatório PDF com <code>reportlab</code> , documentação técnica final do projeto.	Diagnósticos aparecendo na IHM, CSV sendo gravado continuamente, PDF gerado com gráficos e falhas registradas.

8 Cronograma de Entregas

Data	Avaliação	Requisitos
18/05	2ª Avaliação	Simulação no Wokwi ou Proteus com todos os componentes. Hardware virtual funcionando, captura de sinais, IHM local com dados simulados. Todos os requisitos demonstrados em software.
25/05	2ª Avaliação (reposição)	Avaliação teórica para equipes sem projeto funcional.
15/06	3ª Avaliação	Protótipo físico completo e integrado. ESP32 real comunicando com IHM e dashboard IoT. Validação de todos os módulos com transformador real.
22/06	3ª Avaliação (reposição)	Avaliação teórica para equipes sem protótipo funcional.

- **P1:** Circuito de condicionamento simulado com fonte AC virtual no Wokwi;
- **P2:** Firmware base compilando e rodando na simulação;
- **P3:** FFT calculando com dados mockados (seno 120 Hz gerado por software);
- **P4:** Broker Mosquitto ativo, ESP32 publicando dados no tópico correto;
- **P5:** Dashboard exibindo dados em tempo real, mesmo que parcialmente mockados;
- **P6:** Datalogger gravando CSV, pelo menos dois diagnósticos funcionando.

Referências

- IEEE Std C57.91-2011. *IEEE Guide for Loading Mineral-Oil-Immersed Transformers and Step-Voltage Regulators*. IEEE, 2011.

- ABNT NBR 5356-1:2007. *Transformadores de potência — Parte 1: Generalidades*. ABNT, 2007.
- ESPRESSIF SYSTEMS. *ESP32 Technical Reference Manual*. v5.1. Espressif, 2023. Disponível em: https://www.espressif.com/sites/default/files/documentation/esp32_technical_reference_manual_en.pdf.
- OPPENHEIM, A. V.; SCHAFER, R. W. *Discrete-Time Signal Processing*. 3. ed. Pearson, 2009.
- BANKS, J. A. *MQTT Essentials*. Packt Publishing, 2019.
- SWITZER, J. *CT Sensors — Interfacing with an Arduino*. OpenEnergy-Monitor, 2023. Disponível em: <https://learn.openenergymonitor.org/electricity-monitoring/ct-sensors/>.